

Corrections des Examens – 1ère Session

Master 1 Informatique

Université Peleforo Gon Coulibaly

Année académique 2025 – 2026

Table des matières

1 Théorie des Langages Formels et Calculabilité

1.1 Questions de Cours

1. Définition d'une regex et son utilité.

Une **expression régulière (regex)** est une séquence de caractères définissant un motif de recherche. Elle permet de décrire un ensemble de chaînes de caractères de manière concise. Utilités : validation de données, recherche/remplacement de motifs, analyse lexicale en compilation.

2. Définition d'un langage régulier.

Un **langage régulier** est un langage qui peut être décrit par une expression régulière, ou de manière équivalente, reconnu par un automate fini (déterministe ou non). Les langages réguliers sont fermés par union, concaténation, étoile de Kleene, intersection et complément.

3. Définition d'une ε -transition.

Une **ε -transition** (ou transition spontanée) est une transition dans un automate qui se déclenche sans consommer de symbole d'entrée. Elle permet à l'automate de changer d'état sans lire de caractère. Utilisée notamment dans les AFN- ε pour simplifier la construction d'automates.

4. Définition d'un problème de promesse.

Un **problème de promesse** est un problème de décision où l'entrée est restreinte à un sous-ensemble spécifique (la promesse). On ne s'intéresse qu'aux entrées qui satisfont la promesse. Formellement, c'est un couple (P, Q) où P est la promesse et Q la propriété à vérifier.

1.2 Exercices

Exercice 1 : $L = \{a, b, c\}$ sur $\Sigma = \{a, b, c\}$

$$L^2 = L \times L = \{xy \mid x, y \in L\}$$

$$L^2 = \{aa, ab, ac, ba, bb, bc, ca, cb, cc\}$$

Exercice 2 : $L = \{a\}L \cup \{b\}L \cup \{c\}$

Soit une équation linéaire de langage. On cherche le plus petit point fixe.

On peut résoudre par substitution :

$$\text{— } L = aL \cup bL \cup c = (a \cup b)L \cup c$$

$$\text{— Par le lemme d'Arden : } L = (a \cup b)^*c$$

Expression régulière : $(a|b)^*c$

C'est l'ensemble des mots se terminant par c , avec un nombre quelconque (y compris nul) de a et b avant.

Exercice 3 : Grammaire G avec $S \rightarrow aS \mid bA \mid c$ et $A \rightarrow bA \mid c$, $w = aabbc$

Dérivation à gauche :

Dérivation à droite :

$$\begin{aligned} S &\Rightarrow aS \\ &\Rightarrow aaS \\ &\Rightarrow aabA \\ &\Rightarrow aabbA \\ &\Rightarrow aabbc \end{aligned}$$

Exercice 4 : Minimisation d'automate

La méthode de minimisation (Moore) consiste à :

1. Séparer les états en deux classes : états finaux et non-finaux.
2. Partitionner itérativement : deux états sont dans des classes différentes s'ils ont des transitions vers des classes différentes pour un même symbole.
3. S'arrêter quand la partition n'évolue plus.
4. Chaque classe devient un état de l'automate minimal.

2 Théorie de la Complexité

2.1 Questions de Cours

1. Définition d'une instance d'un problème.

Une **instance** d'un problème est une entrée particulière à laquelle on applique l'algorithme de résolution. Par exemple, pour le problème du tri, une instance est un tableau spécifique à trier.

2. Définition de certificat.

Un **certificat** est une preuve succincte permettant de vérifier qu'une solution à un problème est correcte. Pour un problème de décision dans NP, un certificat est une preuve polynomiale qu'une instance est positive.

3. Définition d'un problème NP-Complet.

Un problème est **NP-complet** s'il est :

- Dans NP (vérifiable en temps polynomial)
- NP-difficile (tout problème de NP se réduit polynomialement à lui)

Un des premiers problèmes NP-complets prouvé est SAT (Cook-Levin, 1971).

2.2 Exercices

Exercice 1 : Classement par ordre croissant en $O(\cdot)$

$$f_1(n) = \ln n + \sqrt{n}$$

$$f_2(n) = (\log n)^5 + 2n$$

$$f_3(n) = n \log n$$

$$f_4(n) = n^2 + 10n + 2$$

$$f_5(n) = 3^n + n^4$$

$$f_6(n) = n! + e^{n^2} + 2n$$

Ordre croissant :

$$O(\ln n + \sqrt{n}) < O(n \log n) < O((\log n)^5 + 2n) < O(n^2) < O(3^n) < O(n!)$$

Détail :

- $\ln n + \sqrt{n} = O(\sqrt{n})$ car $\sqrt{n}$ domine $\ln n$
- $n \log n$ est entre linéaire et quadratique
- $(\log n)^5 + 2n = O(n)$ car $2n$ domine $(\log n)^5$
- Donc $n \log n > O(n)$
- $n^2 + 10n + 2 = O(n^2)$
- $3^n + n^4 = O(3^n)$ (exponentiel)
- $n! + e^{n^2} = O(n!)$ (factoriel domine)

Ordre final corrigé :

$$O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(3^n) < O(n!)$$

Soit : $\ln n + \sqrt{n} < (\log n)^5 + 2n < n \log n < n^2 + 10n + 2 < 3^n + n^4 < n! + e^{n^2} + 2n$

Exercice 2 : Complexités $O(\sqrt{n})$ et $O(n^2 \ln n)$, $n = 7800$, 100 op/s

- **Algorithme A** ($O(\sqrt{n})$) : $\sqrt{7800} \approx 88,32$ opérations
- Temps A : $88,32/100 \approx 0,88$ seconde
- **Algorithme B** ($O(n^2 \ln n)$) : $7800^2 \times \ln(7800)$
- $7800^2 = 60\,840\,000$
- $\ln(7800) \approx 8,96$
- $60\,840\,000 \times 8,96 \approx 545\,126\,400$ opérations
- Temps B : $545\,126\,400/100 = 5\,451\,264$ secondes ≈ 63 jours

Exercice 3 : Analyse de complexité en temps de l'Algorithme 1

L'algorithme présenté (Visiter-PP) est un parcours de graphe récursif. La complexité dépend de la structure :

- Chaque nœud est visité une fois
- Pour chaque nœud, on explore ses voisins
- Complexité : $O(|V| + |E|)$ où $|V|$ est le nombre de nœuds et $|E|$ le nombre d'arêtes

Exercice 4 : Analyse de complexité en espace

Complexité spatiale du même algorithme :

- Tableau `couleur` de taille $|V|$
- Tableau `fin` de taille $|V|$
- Pile d'appels récursifs : au pire $|V|$ (chemin linéaire)
- Complexité spatiale : $O(|V|)$

3 Algorithmique Distribuée

3.1 Questions de Cours

1. Qu'est-ce qu'une solution distribuée vivace ?

Une **solution distribuée vivace** est un algorithme distribué garantissant la propriété de **vivacité (liveness)** : tout processus finit par progresser. En exclusion mutuelle, cela signifie que tout processus demandant à entrer en section critique finit par y accéder, même en présence de pannes ou de délais de communication.

2. Problème de l'élection de leader.

L'**élection de leader** est le problème où plusieurs processus dans un système distribué doivent s'accorder pour désigner un unique processus comme coordinateur. Algorithmes célèbres : Bully (ID max), Le Lann (anneau), Chang-Roberts.

Défis : absence de connaissance globale, pannes possibles, topologie variable.

Théorème : impossible sur un anneau anonyme bidirectionnel.

3. Démarche pour écrire un algorithme distribué.

1. **Spécification** : Définir l'objectif, les hypothèses (topologie, synchronisation).
2. **Conception** : Pseudo-code, commandes gardées (Dijkstra), ordinogramme.
3. **Ordonnancement** : Définir le modèle (synchrone ou asynchrone).
4. **Vérification** : Preuve de correction (sûreté + vivacité).
5. **Complexité** : Analyse en nombre de messages et en temps.

4. Fonctionnement d'un adversaire asynchrone aléatoire non-équitable.

Un **adversaire asynchrone** peut décider de l'ordre d'activation des processus. **Aléatoire non-équitable** signifie qu'à chaque pas, il choisit aléatoirement les processus à activer, mais sans garantie d'équité (certains processus peuvent être choisis plus souvent que d'autres). La sélection se fait avec remise selon une loi uniforme : chaque processus u est activé avec probabilité $\frac{1}{|V|}$, et on effectue $|V|$ sélections par pas. Le nombre d'activations suit une loi binomiale $\mathcal{B}(|V|, \frac{1}{|V|})$.

3.2 Exercices

Exercice 1 : Multiplication de deux matrices A (taille m) et B avec 2 processus

Listing 1 – Algorithme distribue de multiplication matricielle

```
1 // Processus P0 (maitre)
2 Entree : matrice A (m x n), matrice B (n x p)
3 Sortie : matrice C (m x p)
4
5 // Division : P0 calcule la premiere moitie, P1 la seconde
6 milieu = m / 2
7
8 // P0 envoie les donnees a P1
9 envoyer(A[milieu+1..m][*], B) a P1
10
11 // P0 calcule sa partie
12 pour i = 1 a milieu faire
13     pour j = 1 a p faire
14         C[i][j] = 0
15         pour k = 1 a n faire
16             C[i][j] = C[i][j] + A[i][k] * B[k][j]
```

```

17         fin pour
18     fin pour
19 fin pour
20
21 // Reception du resultat de P1
22 recevoir(C_partiel) de P1
23 C[milieu+1..m][*] = C_partiel

```

Listing 2 – Processus P1

```

1 // Reception des donnees
2 recevoir(A_partiel, B) de P0
3
4 // Calcul de la partie assignee
5 pour i = 1 a taille(A_partiel) faire
6     pour j = 1 a p faire
7         C[i][j] = 0
8         pour k = 1 a n faire
9             C[i][j] = C[i][j] + A_partiel[i][k] * B[k][j]
10        fin pour
11    fin pour
12 fin pour
13
14 // Envoi du resultat
15 envoyer(C) a P0

```

Exercice 2 : Déterminer le nœud de degré maximal dans une topologie étoile étendue

Listing 3 – Algorithme distribue : noeud de degre maximal

```

1 // Hypothese : topologie en etoile etendue
2 // (un noeud central relie a des sous-centres,
3 //  chaque sous-centre relie a ses feuilles)
4
5 // Phase 1 : Chaque feuille envoie son degre a son sous-centre
6 (Pour chaque feuille) :
7     envoyer(degre = nombre_de_voisins) a son sous-centre
8
9 // Phase 2 : Chaque sous-centre compare les degres recus
10 (Pour chaque sous-centre) :
11     degre_max_local = degre_du_sous_centre
12     pour chaque message reçu d'une feuille :
13         si message.degre > degre_max_local :
14             degre_max_local = message.degre
15             id_max_local = message.id
16     fin pour
17     envoyer(degre_max_local, id_max_local) au centre
18
19 // Phase 3 : Le centre determine le maximum global
20 (Pour le centre) :
21     degre_max_global = degre_du_centre
22     pour chaque message reçu d'un sous-centre :
23         si message.degre > degre_max_global :
24             degre_max_global = message.degre
25             id_max_global = message.id

```

```
26     fin pour
27     // Diffusion du resultat a tous les noeuds
28     diffuser(noeud_max = id_max_global, degre = degre_max_global)
```


4 Programmation Avancée en C++

4.1 Questions

1. Différences fondamentales entre structures et classes en C++.

struct	class
Membres publics par défaut	Membres privés par défaut
Héritage public par défaut	Héritage privé par défaut
Utilisé pour POD (Plain Old Data)	Utilisé pour l'encapsulation
Peut avoir méthodes/constructeurs	Même capacité

Remarque : En C++, les structures sont techniquement identiques aux classes sauf pour les valeurs par défaut.

2. Sérialisation.

La **sérialisation** est le processus de conversion d'un objet en un format (binaire, XML, JSON) pouvant être stocké ou transmis. La désérialisation est l'opération inverse. Utile pour : persistance, communication réseau, échange de données entre programmes.

4.2 Exercices

3. Programme pour déterminer le degré moyen d'un graphe.

Listing 4 – Degré moyen d'un graphe

```
1 #include <iostream>
2 #include <vector>
3
4 double degreMoyen(const std::vector<std::vector<int>>& graphe) {
5     int totalDegres = 0;
6     int nbSommets = graphe.size();
7
8     for (int i = 0; i < nbSommets; i++) {
9         totalDegres += graphe[i].size();
10    }
11
12    return (double)totalDegres / nbSommets;
13 }
14
15 int main() {
16     // Exemple : graphe non-orienté (liste d'adjacence)
17     std::vector<std::vector<int>> graphe = {
18         {1, 2},      // sommet 0
19         {0, 2, 3},   // sommet 1
20         {0, 1},      // sommet 2
21         {1}          // sommet 3
22     };
23
24     std::cout << "Degré_moyen : " << degreMoyen(graphe) << std::endl;
25     return 0;
26 }
```

4. Programme pour multiplier trois matrices carrées.

Listing 5 – Multiplication de trois matrices

```
1 #include <iostream>
```

```

2  #include <vector>
3
4  using Matrice = std::vector<std::vector<int>>>;
5
6  Matrice multiplier(const Matrice& A, const Matrice& B) {
7      int n = A.size();
8      Matrice C(n, std::vector<int>(n, 0));
9
10     for (int i = 0; i < n; i++)
11         for (int j = 0; j < n; j++)
12             for (int k = 0; k < n; k++)
13                 C[i][j] += A[i][k] * B[k][j];
14
15     return C;
16 }
17
18 Matrice multiplierTroisMatrices(const Matrice& A,
19                                 const Matrice& B,
20                                 const Matrice& C) {
21     // (A * B) * C ou A * (B * C)
22     Matrice AB = multiplier(A, B);
23     return multiplier(AB, C);
24 }

```

5. Point d'intersection de deux segments.

Listing 6 – Intersection de deux segments

```

1  #include <iostream>
2
3  struct Point {
4      double x, y;
5      Point(double x = 0, double y = 0) : x(x), y(y) {}
6  };
7
8  int orientation(const Point& p, const Point& q, const Point& r) {
9      double val = (q.y - p.y) * (r.x - q.x) -
10                 (q.x - p.x) * (r.y - q.y);
11      if (val == 0) return 0;    // colineaire
12      return (val > 0) ? 1 : 2; // sens horaire ou antihoraire
13  }
14
15  bool pointSurSegment(const Point& p, const Point& q, const Point& r) {
16      return (q.x <= std::max(p.x, r.x) && q.x >= std::min(p.x, r.x) &&
17              q.y <= std::max(p.y, r.y) && q.y >= std::min(p.y, r.y));
18  }
19
20  bool segmentsIntersect(Point p1, Point q1, Point p2, Point q2) {
21      int o1 = orientation(p1, q1, p2);
22      int o2 = orientation(p1, q1, q2);
23      int o3 = orientation(p2, q2, p1);
24      int o4 = orientation(p2, q2, q1);
25
26      if (o1 != o2 && o3 != o4) return true;
27  }

```

```

28 // Cas colineaires
29 if (o1 == 0 && pointSurSegment(p1, p2, q1)) return true;
30 if (o2 == 0 && pointSurSegment(p1, q2, q1)) return true;
31 if (o3 == 0 && pointSurSegment(p2, p1, q2)) return true;
32 if (o4 == 0 && pointSurSegment(p2, q1, q2)) return true;
33
34 return false;
35 }

```

6. Fonction générique pour vérifier la connexité d'un graphe.

Listing 7 – Connexité d'un graphe

```

1  #include <iostream>
2  #include <vector>
3  #include <stack>
4
5  template<typename T>
6  bool estConnexe(const std::vector<std::vector<T>>& graphe) {
7      int n = graphe.size();
8      if (n == 0) return true;
9
10     std::vector<bool> visite(n, false);
11     std::stack<int> pile;
12
13     pile.push(0);
14     visite[0] = true;
15
16     while (!pile.empty()) {
17         int sommet = pile.top();
18         pile.pop();
19
20         for (int voisin : graphe[sommet]) {
21             if (!visite[voisin]) {
22                 visite[voisin] = true;
23                 pile.push(voisin);
24             }
25         }
26     }
27
28     for (bool v : visite)
29         if (!v) return false;
30
31     return true;
32 }

```

5 Ingénierie des Protocoles

5.1 Questions de Cours

1. Démarche pour concevoir un protocole.

1. **Spécification des besoins** : objectifs, contraintes, environnement.
2. **Définition des services** : types de messages, primitives.
3. **Conception architecturale** : topologie, hiérarchie, couches.
4. **Spécification formelle** : machine à états, algèbre de processus.
5. **Implémentation** : codage du protocole.
6. **Vérification** : tests, simulation, preuve formelle.
7. **Déploiement et maintenance**.

2. Éléments constitutifs d'un protocole.

- **Syntaxe** : format des messages, encodage.
- **Sémantique** : signification des messages, actions associées.
- **Temporisation** : délais, timeouts, échéances.
- **Mécanismes de fiabilité** : accusés de réception, retransmissions.
- **Ordonnancement** : séquencement des échanges.

3. Techniques de commutation.

- **Commutation de circuits** : réservation d'un chemin dédié (téléphone).
- **Commutation de messages** : stockage et transmission intégrale.
- **Commutation de paquets** : découpage en paquets, routage individuel.
- **Commutation de cellules** : paquets de taille fixe (ATM).

5.2 Cas Pratique

Conception d'un protocole pour synchroniser les horloges dans un réseau.

Listing 8 – Protocole de synchronisation d'horloges

```
1 // Protocole inspire de NTP simplifié
2 Structure MessageSync {
3     horodatage_depart: timestamp // T1 : moment de l'envoi du message
4     horodatage_arrivee: timestamp // T2 : reception par le serveur
5     horodatage_retour: timestamp // T3 : renvoi par le serveur
6 }
7
8 // Client (machine asservie)
9 Procedure synchroniser(serveur) :
10     T1 = horloge_locale()
11     envoyer MessageSync(T1, ?, ?) a serveur
12     recevoir MessageSync(T1, T2, T3) de serveur
13     T4 = horloge_locale()
14
15     // Calcul du delai aller-retour (RTT)
16     RTT = (T4 - T1) - (T3 - T2)
17
18     // Calcul du decalage
```

```

19     decalage = ((T2 - T1) + (T3 - T4)) / 2
20
21     // Ajustement de l'horloge locale
22     horloge_locale() = horloge_locale() + decalage
23 Fin Procedure
24
25 // Serveur (reference)
26 A la reception de MessageSync(T1, ?, ?) de client :
27     T2 = horloge_reference()
28     // Traitement
29     T3 = horloge_reference()
30     envoyer MessageSync(T1, T2, T3) a client
31 Fin Procedure

```

6 Simulation Informatique

6.1 Questions de Cours

1. Démarche pour conduire un projet de simulation.

1. **Formulation du problème** : objectifs, hypothèses.
2. **Collecte des données** : paramètres réels, distributions.
3. **Conception du modèle** : entités, variables, relations.
4. **Implémentation** : codage du simulateur.
5. **Vérification et validation** : le modèle est-il correct ?
6. **Expérimentation** : exécution des scénarios.
7. **Analyse des résultats** : interprétation statistique.
8. **Documentation et décision**.

2. Définition d'un scénario.

Un **scénario** est un ensemble de paramètres et de conditions décrivant une situation spécifique à simuler. Il définit les valeurs des facteurs contrôlés (nombre de nœuds, intensité du trafic, etc.) pour une exécution particulière de la simulation.

3. Notion de bruit blanc.

Le **bruit blanc** est un processus aléatoire dont la valeur à chaque instant est indépendante et identiquement distribuée (bruit non corrélé). En simulation, il modélise les perturbations aléatoires. Sa densité spectrale de puissance est constante sur toutes les fréquences.

4. Simulation à événements discrets.

La **simulation à événements discrets** (DES) modélise un système où les changements d'état se produisent à des instants précis. L'horloge de simulation saute d'un événement au suivant. Chaque événement est horodaté et traité dans l'ordre chronologique. Utilisée pour : réseaux, files d'attente, systèmes de production.

6.2 Cas Pratique

Contexte : Évaluation de deux protocoles d'équilibrage de charge (A et B) dans un réseau sans fil.

1. Identification

- **Entités** : Nœuds du réseau ($N = 100, 200, 300, 400, 500$), serveur, canaux radio.
- **Événements** : Arrivée de paquet, transmission, handover, perte de paquet, fin de transmission.
- **Variables d'état** : Taille de la file d'attente, état du canal (occupé/libre), SNR courant, position des nœuds.
- **Phénomènes stochastiques** : Arrivée des paquets (processus de Poisson), SNR (variable aléatoire), mobilité des nœuds, erreurs de transmission.

2. Nombre d'essais

Chaque scénario répété 40 fois.

Nombre de combinaisons : $5 \times 3 \times 3 = 45$ scénarios (N : 5 valeurs, λ : 3, SNR : 3).

Nombre total d'essais : $45 \times 40 = 1800$ essais.

3. Intérêt des répétitions

Les répétitions permettent :

- D'obtenir une **moyenne statistique** fiable
- De **réduire la variance** des résultats
- De calculer des **intervalles de confiance**
- D'éliminer les **effets aléatoires** d'une seule exécution

4. Générateurs pseudo-aléatoires

- **Trafic** : Loi de Poisson pour les arrivées, loi exponentielle pour les inter-arrivées.
- **Mobilité** : Modèle Random Waypoint, Gauss-Markov.
- **Erreurs radio** : Taux d'erreur binaire (BER) fonction du SNR, modèle de Gilbert-Elliot.

5. Début de collecte des données

La collecte doit débiter après la **période de chauffe (warm-up)**. On peut la déterminer par :

- Observation de la stabilité de la moyenne
- Test de convergence (ex : Welch)
- Généralement après que le régime permanent est atteint

6. Débit moyen théorique

Données : $\lambda_{\max} = 100$ paquets/s, $L = 1000$ bits

$$D_{\text{th}} = \lambda \times L = 100 \times 1000 = 100\,000 \text{ bps} = 100 \text{ kbps}$$

7. Cohérence du résultat $D_{\text{sim}} = 450 \text{ kbps}$

$D_{\text{sim}} = 450 \text{ kbps}$ et $D_{\text{th}} = 100 \text{ kbps}$.

Le résultat de simulation (450 kbps) est **incohérent** car il dépasse largement la limite théorique maximale de 100 kbps. Causes possibles :

- Erreur dans le modèle de simulation
- Paramètres mal configurés
- Unité incorrecte dans la mesure

Conclusion : Le modèle doit être revu et corrigé avant exploitation.

7 Programmation Parallèle (OpenMP)

7.1 Questions de Cours

1. Principes fondamentaux de la programmation parallèle.

- **Parallélisme de tâches** : répartition de différentes tâches sur plusieurs cœurs.
- **Parallélisme de données** : distribution des données entre processeurs.
- **Synchronisation** : coordination de l'accès aux ressources partagées.
- **Passage à l'échelle** : performance proportionnelle au nombre de cœurs.
- **Équilibrage de charge** : répartition équitable du travail.

2. Qu'est-ce qu'un fil d'exécution ?

Un **thread** (ou fil d'exécution) est la plus petite unité de traitement pouvant être ordonnée par le système d'exploitation. Un processus peut contenir plusieurs threads partageant le même espace mémoire, permettant une exécution parallèle sur des cœurs multiples.

3. Types de parallélisme selon l'objet distribué.

- **Parallélisme de données** : même opération sur des données différentes (SIMD).
- **Parallélisme de tâches** : différentes opérations sur des données identiques ou différentes.
- **Parallélisme de pipeline** : données traversant une séquence d'étapes de traitement.
- **Parallélisme de contrôle** : branches conditionnelles exécutées en parallèle.

7.2 Exercice 2 : Correction du programme OpenMP

Listing 9 – Programme corrige

```
1  #include <iostream>
2  #include "omp.h"
3  using namespace std;
4
5  int main()
6  {
7      int N = 1000;
8      double **A, **B;
9
10     A = new double*[N];
11     B = new double*[N];
12
13     // CORRECTION 1 : i < N au lieu de i <= N
14     for (int i = 0; i < N; i++)
15     {
16         A[i] = new double[N];
17         B[i] = new double[N];
18     }
19
20     #pragma omp parallel for
21     for (int i = 0; i < N; i++)
22     {
23         // CORRECTION 2 : #pragma omp for au lieu de #pragma omp
                parallel for
24         // (on est déjà dans une region parallele)
```



```

25     #pragma omp for
26     for (int j = 0; j < N; j++)
27     {
28         A[i][j] = i + j;
29     }
30 }
31
32 #pragma omp parallel
33 {
34     #pragma omp for collapse(2)
35     for (int i = 0; i < N; i++)
36     {
37         for (int j = 0; j < N; j++)
38         {
39             B[i][j] = A[i][j];
40         }
41     }
42 }
43
44 cout << B[0][1] << endl;
45
46 for (int i = 0; i < N; i++)
47 {
48     delete[] A[i];
49     delete[] B[i];
50 }
51 delete[] A;
52 delete[] B;
53
54 return 0;
55 }

```

Erreurs corrigées :

1. **Ligne 12** : $i \leq N \rightarrow i < N$ pour éviter un débordement de tableau (accès à $A[N]$ qui n'existe pas).
2. **Ligne 17** : `#pragma omp parallel for` $\rightarrow$ `#pragma omp for` car on est déjà dans une région parallèle créée par la directive `#pragma omp parallel for` de la ligne 16. Une nouvelle région parallèle imbriquée gaspille des threads et peut causer une surcharge.

Exercice 3 : Tri d'un tableau distribué avec OpenMP

Listing 10 – Tri parallele (Tri par fusion) avec OpenMP

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <omp.h>
5
6  void triFusion(std::vector<int>& tab, int debut, int fin) {
7      if (debut < fin) {
8          int milieu = (debut + fin) / 2;
9
10         #pragma omp task shared(tab)
11         triFusion(tab, debut, milieu);

```

```

12
13     #pragma omp task shared(tab)
14     triFusion(tab, milieu + 1, fin);
15
16     #pragma omp taskwait
17     std::inplace_merge(tab.begin() + debut,
18                       tab.begin() + milieu + 1,
19                       tab.begin() + fin + 1);
20 }
21 }
22
23 void trierTableauDistribue(std::vector<int>& tab, int nbProcessus) {
24     int taille = tab.size();
25     int segment = taille / nbProcessus;
26
27     #pragma omp parallel num_threads(nbProcessus)
28     {
29         int id = omp_get_thread_num();
30         int debut = id * segment;
31         int fin = (id == nbProcessus - 1) ? taille - 1 : debut +
            segment - 1;
32
33         std::sort(tab.begin() + debut, tab.begin() + fin + 1);
34     }
35
36     // Fusion des segments tries
37     #pragma omp parallel
38     {
39         #pragma omp single
40         triFusion(tab, 0, taille - 1);
41     }
42 }

```

Exercice 4 : Programme MPI pour afficher le rang des processus

Listing 11 – Affichage du rang avec MPI

```

1  #include <iostream>
2  #include <mpi.h>
3
4  int main(int argc, char** argv) {
5      MPI_Init(&argc, &argv);
6
7      int rang, nbProcessus;
8      MPI_Comm_rank(MPI_COMM_WORLD, &rang);
9      MPI_Comm_size(MPI_COMM_WORLD, &nbProcessus);
10
11      std::cout << "Processus_" << rang << "_sur_"
12                << nbProcessus << "_processus" << std::endl;
13
14      MPI_Finalize();
15      return 0;
16 }

```

Compilation et exécution :

```
1 mpicxx -o programme programme.cpp
2 mpirun -np 4 ./programme
```